

Tarea 02: Algoritmos, Diagramas de Flujo y Estructuras de Control

Maryom Alejandra Vicente González

Carné: 202202918

Escuela de Ingeniería Mecánica Eléctrica

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Resumen—En esta tarea se aplicaron algoritmos y diagramas de flujo mediante PSeInt, además de estructuras de control en GNU Octave y Python. Se trabajaron condiciones, ciclos, operaciones con matrices y se configuraron funciones para observar su funcionamiento y comparar su sintaxis.

I. ÁREA DEL CÍRCULO

I-A. Algoritmo

1. Iniciar.
2. Solicitar el radio del círculo.
3. Leer el radio.
4. Calcular el área mediante $3,14 \times \text{radio} \times \text{radio}$.
5. Mostrar el resultado.
6. Finalizar.

I-B. Diagrama de flujo

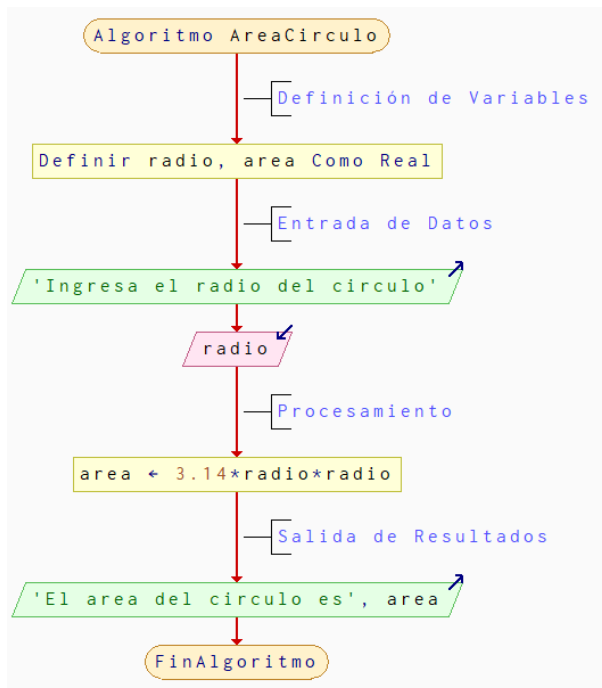


Figura 1. Diagrama de flujo para calcular el área de un círculo.

I-C. Código en PSeInt

```
Proceso AreaCirculo

    Definir radio, area Como Real;

    Escribir "Ingresa el radio del círculo";
    Leer radio;

    area = 3.14 * radio * radio;

    Escribir "El area del círculo es ", area;

FinProceso
```

I-D. Resultado

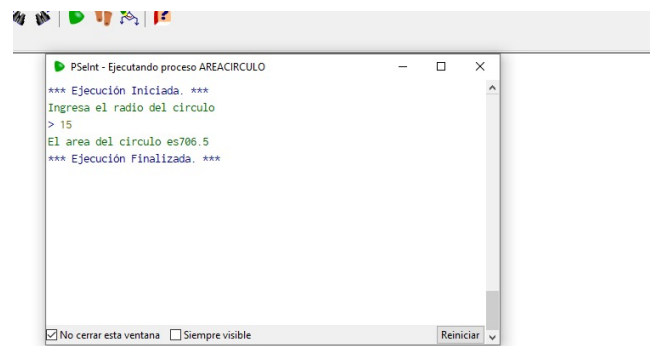


Figura 2. Resultado del cálculo del área en PSeInt.

II. ESTRUCTURA CONDICIONAL

II-A. Algoritmo

1. Asignar valores a las variables x y y .
2. Comparar si x es mayor que y .
3. Si se cumple, mostrar que x es mayor.
4. Si no se cumple, comprobar si ambos valores son iguales.
5. En caso contrario, mostrar que y es mayor.
6. Finalizar.

II-B. Diagrama de flujo

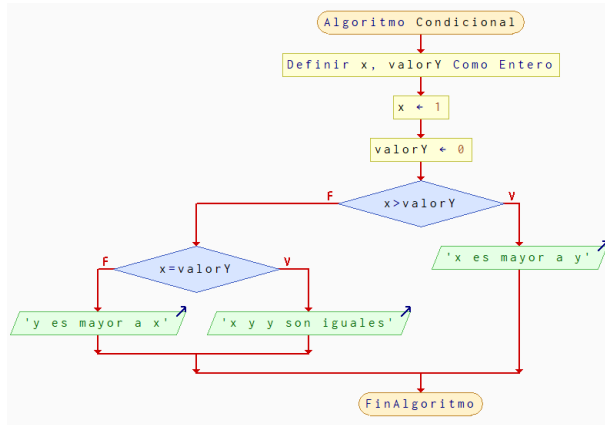


Figura 3. Diagrama de flujo de la estructura condicional.

II-C. Código en Octave

```
x = 1
y = 0

if (x > y)
    'x es mayor a y'
elseif (x == y)
    'x y y son iguales'
else
    'y es mayor a x'
endif
```

II-D. Código equivalente en Python

```
x = 1
y = 0

if x > y:
    print("x es mayor a y")
elif x == y:
    print("x y y son iguales")
else:
    print("y es mayor a x")
```

II-E. Resultado

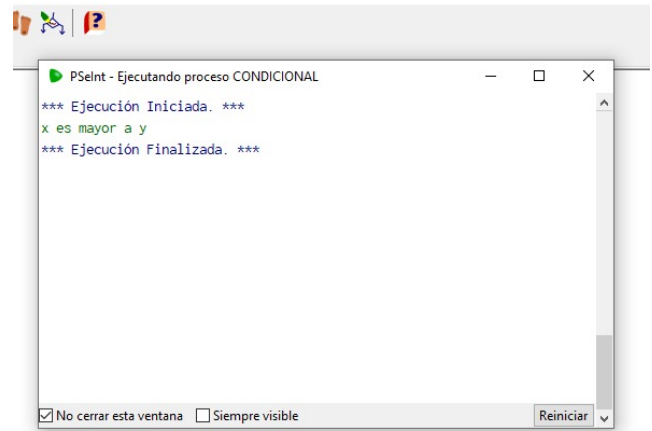


Figura 4. Resultado de la estructura condicional.

III. CONDICIÓN COMPUESTA

III-A. Algoritmo

1. Asignar valores a x , y y z .
2. Comprobar si $x > y$ y $z < 0$.
3. Mostrar el mensaje correspondiente si ambas condiciones son verdaderas.
4. En caso contrario, comprobar una segunda condición utilizando OR.
5. Mostrar el mensaje correspondiente.
6. Finalizar.

III-B. Diagrama de flujo

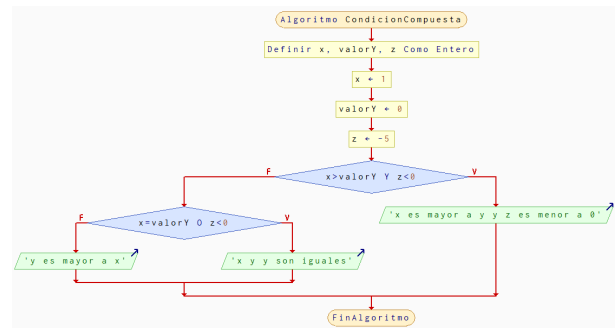


Figura 5. Diagrama de flujo de una condición compuesta.

III-C. Código en Octave

```
x = 1
y = 0
z = -5

if (x > y & z < 0)
    'x es mayor a y y z es menor a 0'
elseif (x == y | z < 0)
```

```

    'x y y son iguales'
else
    'y es mayor a x'
endif

```

III-D. Código equivalente en Python

```

x = 1
y = 0
z = -5

if x > y and z < 0:
    print("x es mayor a y y z es menor a 0")
elif x == y or z < 0:
    print("x y y son iguales")
else:
    print("y es mayor a x")

```

III-E. Resultado

```

*** Ejecución Iniciada. ***
x es mayor a y y z es menor a 0
*** Ejecución Finalizada. ***

```

Figura 6. Resultado de la condición compuesta.

IV. CICLO WHILE

IV-A. Algoritmo

1. Asignar $y = 0$ y $z = -5$.
2. Comprobar si z es menor que y .
3. Mostrar el valor actual de z .
4. Incrementar z en una unidad.
5. Repetir mientras $z < y$.
6. Finalizar.

IV-B. Diagrama de flujo

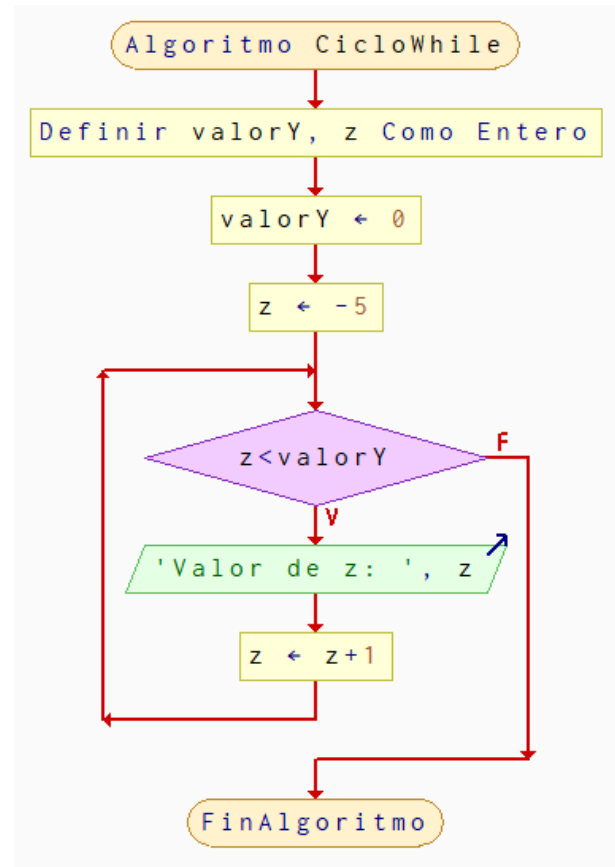


Figura 7. Diagrama de flujo correspondiente al ciclo While.

IV-C. Código en Octave

```

x = 1
y = 0
z = -5

while (z < y)

    'Valor de z'
    z

    ++z

endwhile

```

IV-D. Código equivalente en Python

```

x = 1
y = 0
z = -5

while z < y:

    print("Valor de z:", z)

    z += 1

```

IV-E. Resultado

```
*** Ejecución Iniciada. ***  
Valor de z: -5  
Valor de z: -4  
Valor de z: -3  
Valor de z: -2  
Valor de z: -1  
*** Ejecución Finalizada. ***
```

Figura 8. Resultado obtenido del ciclo While.

V. CICLO FOR: FIBONACCI

V-A. Algoritmo

1. Crear un vector de diez posiciones.
2. Inicializar los primeros valores de la sucesión.
3. Iniciar el ciclo desde la tercera posición.
4. Sumar los dos elementos anteriores.
5. Guardar el resultado en la posición actual.
6. Repetir hasta completar las diez posiciones.
7. Mostrar la sucesión.

V-B. Diagrama de flujo

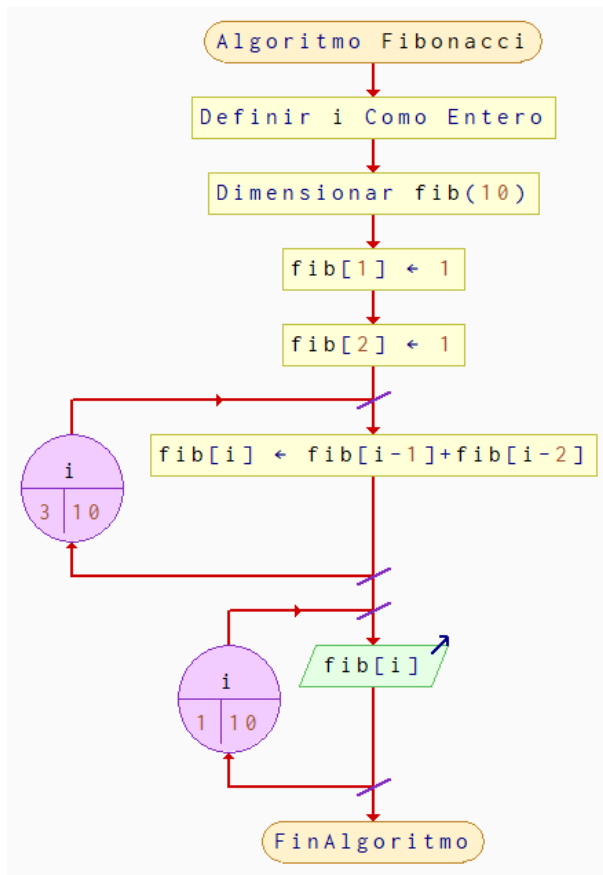


Figura 9. Diagrama de flujo para generar la sucesión de Fibonacci.

V-C. Código en Octave

```
fib = ones(1,10)  
  
for i = 3:10  
    fib(i) = fib(i-1) + fib(i-2);  
endfor  
  
fib
```

V-D. Código equivalente en Python

```
fib = [1] * 10  
  
for i in range(2,10):  
    fib[i] = fib[i-1] + fib[i-2]  
  
print(fib)
```

V-E. Resultado

```
*** Ejecución Iniciada. ***  
1  
1  
2  
3  
5  
8  
13  
21  
34  
55  
*** Ejecución Finalizada. ***
```

Figura 10. Resultado de la sucesión de Fibonacci.

VI. MANEJO DE ERRORES

VI-A. Algoritmo

1. Intentar ejecutar una operación.
2. Comprobar si ocurre un error.
3. Si ocurre un error, mostrar un mensaje.
4. Continuar con la ejecución normal.
5. Finalizar.

VI-B. Diagrama de flujo

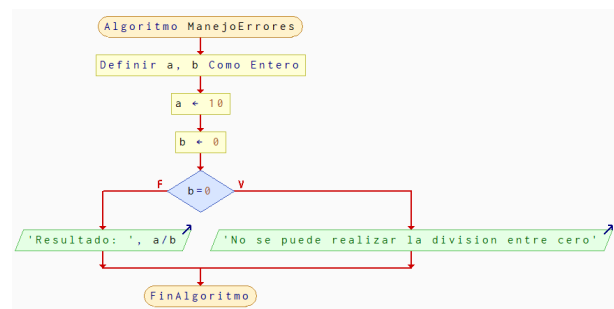


Figura 11. Diagrama de flujo del manejo de errores.

VI-C. Código en Octave

```
try
    m = [1:5;10:15]
catch
    'NO SE PUDO EJECUTAR, se continua con la ejecucion normal
end_try_catch
```

VI-D. Código equivalente en Python

```
try:
    fila1 = list(range(1,6))
    fila2 = list(range(10,16))

    if len(fila1) != len(fila2):
        raise ValueError

except ValueError:
    print("NO SE PUDO EJECUTAR")
```

VI-E. Resultado

```
*** Ejecución Iniciada. ***
No se puede realizar la division entre cero
*** Ejecución Finalizada. ***
```

Figura 12. Resultado del manejo de errores.

VII. OPERACIONES CON MATRICES

VII-A. Algoritmo

1. Crear las matrices M y N .
2. Realizar la suma de las matrices.
3. Realizar la resta.
4. Realizar la multiplicación.
5. Obtener el producto punto.
6. Obtener la transpuesta de una matriz.
7. Mostrar los resultados.

VII-B. Diagrama de flujo

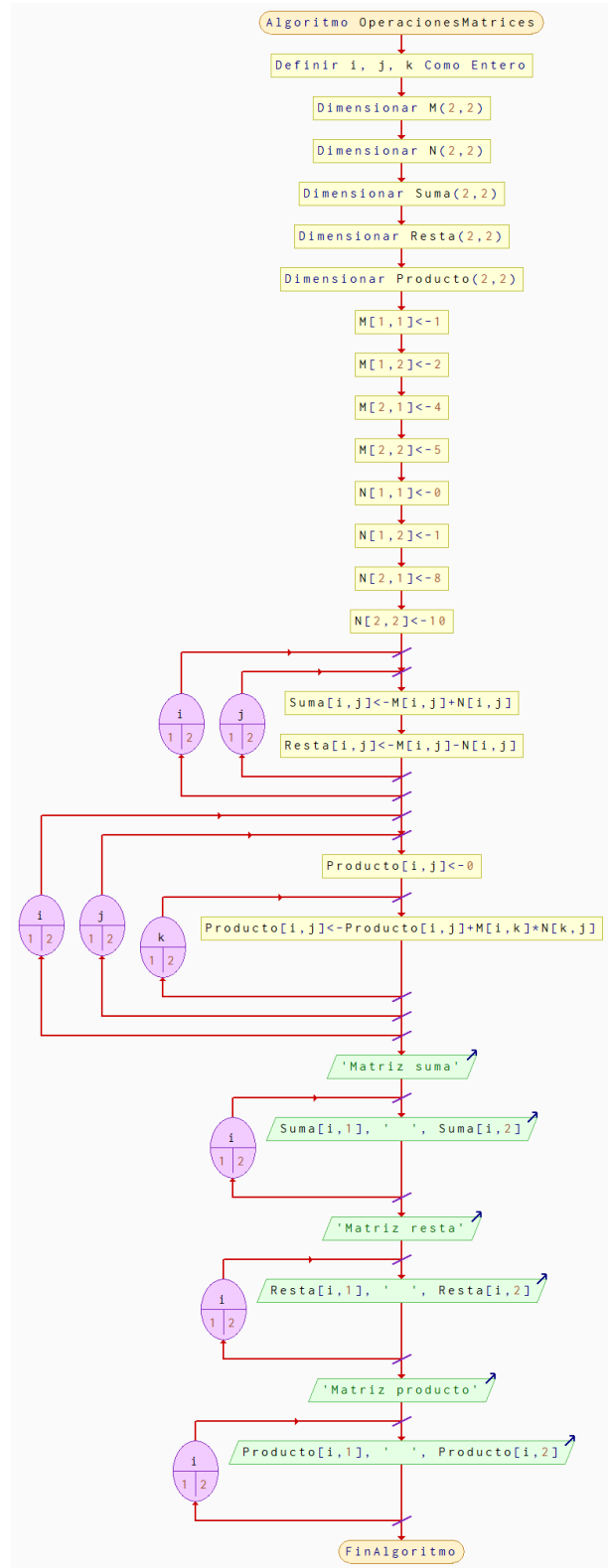


Figura 13. Diagrama de flujo de las operaciones con matrices.

VII-C. Código en Octave

```
M = [1,2;4,5]
N = [0,1;8,10]
```

```
M + N
M - N
M * N
```

```
dot(M,N)
```

```
M'
```

VII-D. Código equivalente en Python

```
import numpy as np

M = np.array([
    [1,2],
    [4,5]
])

N = np.array([
    [0,1],
    [8,10]
])

print(M + N)

print(M - N)

print(M @ N)

print(np.sum(M * N))

print(M.T)
```

VII-E. Resultado

```
*** Ejecución Iniciada. ***
Matriz suma
1 3
12 15
Matriz resta
1 1
-4 -5
Matriz producto
16 21
40 54
*** Ejecución Finalizada. ***
```

Figura 14. Resultados de las operaciones con matrices.

VIII. FUNCIONES Y GRÁFICAS

VIII-A. GNU Octave

Se realizaron pruebas con funciones y diferentes tipos de gráficas en GNU Octave.

La función utilizada fue:

```
function y = funcion(x)
    y = 4*x.^3 + 10*x.^2 + 6;
endfunction
```

Para generar una gráfica discreta se utilizó la función stem:

```
x = linspace(-3,1,50);
stem(x,funcion(x));
title('Titulo');
xlabel('Eje X');
ylabel('Eje Y');
```

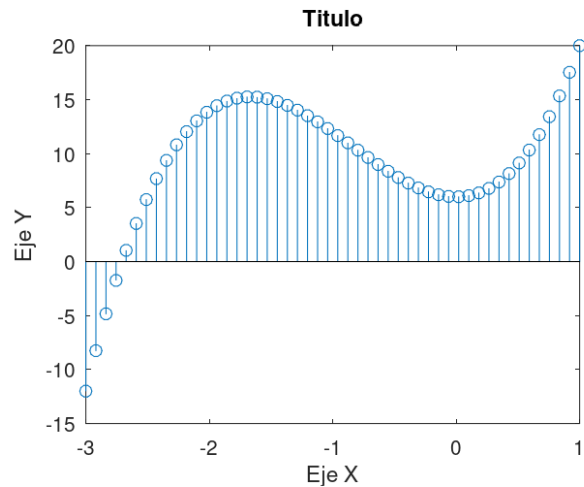


Figura 15. Gráfica utilizando stem en GNU Octave.

También se graficaron las funciones seno y coseno en una misma figura.

```
x = 0:0.1:4*pi;

y1 = sin(x);
y2 = cos(x);

hold on;

p1 = plot(x,y1);
p2 = plot(x,y2);

set(p1,'Color','red','LineWidth',2);
set(p2,'Color','blue','LineWidth',1);

xlabel('Eje X');
ylabel('Eje Y');
title('Seno y Coseno');
legend('Seno','Coseno');

hold off;
```

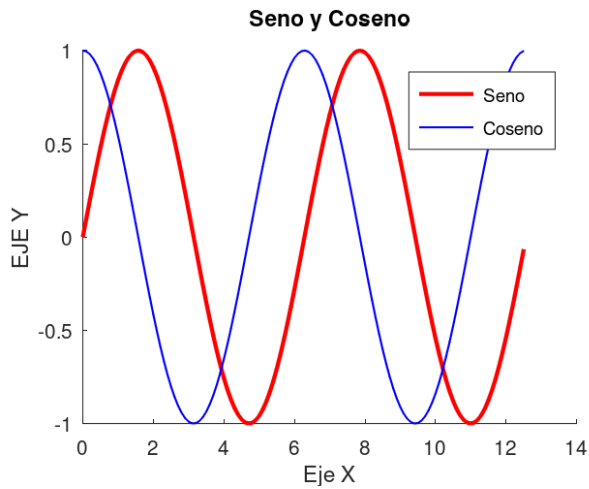


Figura 16. Gráficas de seno y coseno en GNU Octave.

VIII-B. Python

Los ejemplos anteriores se realizaron también en Python utilizando NumPy y Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

def funcion(x):
    return 4*x**3 + 10*x**2 + 6

x = np.linspace(-3,1,50)
y = funcion(x)

plt.stem(x,y)
plt.title("Titulo")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.show()
```

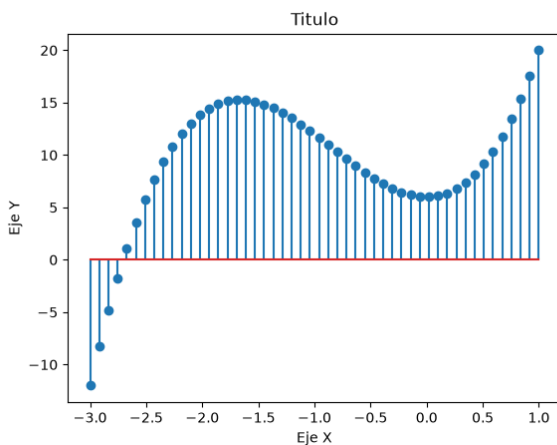


Figura 17. Gráfica utilizando stem en Python.

Para las funciones seno y coseno se utilizó:

```
x = np.arange(0, 4*np.pi, 0.1)
y1 = np.sin(x)
y2 = np.cos(x)
plt.plot(x, y1,
```

```
color="red",
linewidth=2,
label="Seno")

plt.plot(x, y2,
color="blue",
linewidth=1,
label="Coseno")

plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.title("Seno y Coseno")
plt.legend()
plt.show()
```

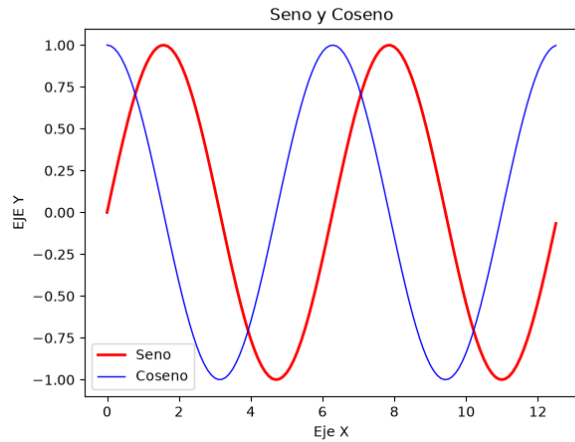


Figura 18. Gráficas de seno y coseno en Python.

VIII-C. Resultado de la conexión

En PostgreSQL se verificó que la tabla `redes` contenía correctamente el registro almacenado.

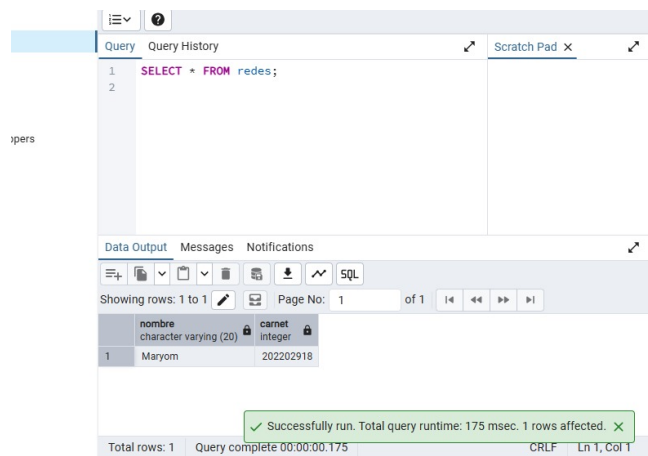
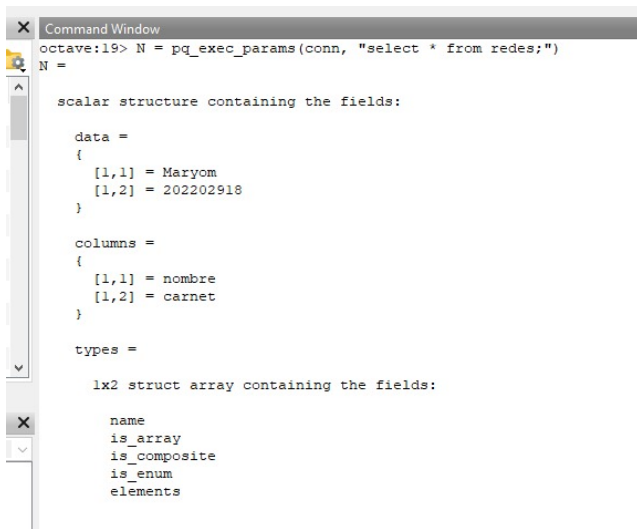


Figura 19. Registro almacenado en la tabla `redes` de PostgreSQL.

Posteriormente, desde GNU Octave se realizó una consulta a la misma tabla mediante la conexión establecida con PostgreSQL.



```
octave:19> N = pg_exec_params(conn, "select * from redes;")
N =

scalar structure containing the fields:

data =
{
  [1,1] = Maryom
  [1,2] = 202202918
}

columns =
{
  [1,1] = nombre
  [1,2] = carnet
}

types =

1x2 struct array containing the fields:

name
is_array
is_composite
is_enum
elements
```

Figura 20. Consulta de la tabla redes realizada desde GNU Octave.

La consulta permitió comprobar que GNU Octave pudo conectarse correctamente con PostgreSQL y obtener los datos almacenados en la base de datos.

VIII-D. Repositorio

El código desarrollado durante la actividad se encuentra disponible en el siguiente repositorio:

<https://github.com/3050321780117-coder/Repopruebamaryom>

IX. RESULTADOS GENERALES

Se realizaron correctamente las pruebas de algoritmos, estructuras de control, matrices y funciones utilizando PSeInt, GNU Octave y Python. También se generaron diferentes gráficas, comprobando que Octave y Python permiten obtener resultados similares mediante instrucciones con distinta sintaxis.

X. CONCLUSIONES

- Los algoritmos y diagramas de flujo ayudaron a organizar cada ejercicio.
- Se practicó el uso de condicionales, ciclos, matrices y funciones.
- Octave y Python obtuvieron resultados similares con diferente sintaxis.